

Ficha de atividade 0

O Ambiente de Desenvolvimento

Esta primeira aula tem como objetivo garantir que todos possuem o ambiente que será usado nos trabalhos práticos corretamente instalado e configurado.

Na componente teórico-prática, os alunos poderão usar a linguagem de programação na qual estejam mais confortáveis. Contudo, os exemplos e enunciados usarão como base a linguagem de programação `Python` (versão 3.x). As instruções para a instalação e configuração básicas podem ser encontradas neste [link](#).

Programa de Exemplo

Para testar o ambiente de desenvolvimento e a instalação do `Python`, escreva um pequeno programa que testa se um inteiro passado como parâmetro é um número primo. Exemplos:

```
$ python primo.py 20
False

$ python primo.py 21
True
```

Instalação de bibliotecas de suporte

Cryptography - <https://cryptography.io/en/latest/>

A biblioteca criptográfica que iremos usar maioritariamente no curso é `cryptography`. Trata-se de uma biblioteca para a linguagem Python bem desenhada e bem documentada que oferece uma API de alto nível a diferentes "Serviços Criptográficos" (*recipes*). No entanto, e no âmbito concreto deste curso, iremos fazer um uso "menos standard" dessa biblioteca, na medida em que iremos recorrer directamente à funcionalidade de baixo nível.

Instalação:

Sugere-se o método de instalação baseado no `pip` (ver no [link](#).)

```
$ pip3 install --upgrade pip
$ pip3 install cryptography
```

Comunicação segura entre cliente-servidor em Python

Ao longo do semestre, usaremos diferentes criptosistemas para proteger propriedades de segurança na comunicação entre dois processos (locais ou remotos).

As scripts `Client.py` e `Server.py`, disponíveis na área de conteúdos da blackboard, constituem uma implementação elementar de uma aplicação que permite a um número arbitrário de clientes comunicar com um servidor que escuta numa dada porta TCP (e.g. 7777). O servidor atribui um número de ordem a cada cliente, e simplesmente faz o *dump* do texto enviado por este cliente (prefixando cada linha com o respectivo número de ordem). Quando um cliente fecha a ligação, o servidor assinala o facto (e.g. imprimindo [n], onde *n* é o número do cliente).

Exemplo da execução do servidor (que comunica com 3 clientes):

```
$ python3 Servidor.py
1 : daskj djdhs slfghfjs askj
1 : asdkdh fdhss
1 : sjd
```

```
2 : iidhs
2 : asdjhf sdga
2 : sadjjd d dhhsj
3 : djsh
1 : sh dh d d
3 : jdhd kasjdh as
2 : dsaj dasjh
3 : asdj dhdhsjsh
[3]
2 : sjdh
1 : dhgd ss
[1]
2 : djdj
[2]
```